MochaCat Studio — 25 Senior SQL, Python & System Design Production Traps (2026 Cheat Sheet)

> **Official Free Companion to the MochaCat Senior Engineering Quiz Series**
> **Subscribe on YouTube:** https://www.youtube.com/channel/UCW6R0z_vBpQX0lqDLJOmslQ?sub_confirmation=1
> **Unlock the Complete 30-Blueprint Playbook + Runnable Code ($19) or Full Bundle ($29):**
> https://gabhack.github.io/mochacat-store/

Part 1 — PostgreSQL & Production SQL Traps (#1–#8)

1. The `NOT IN (..., NULL)` Silent Zero-Row Trap

- **The Bug:** `SELECT * FROM users WHERE id NOT IN (1, 2, NULL);` returns **0 rows**!
- **Why:** `id != NULL` evaluates to `UNKNOWN` in three-valued SQL logic, collapsing the entire `AND` chain.
- **Senior Fix:** Always use `WHERE NOT EXISTS (...)` or filter `WHERE sub.id IS NOT NULL`.

2. Why `SELECT *` Kills Index-Only Scans at Scale

- **The Bug:** Table has 20M rows + 40 columns and a fast covering index on `(user_id, total_cents)`, yet `SELECT *` jumps from `0.8ms` to `180ms`.
- **Why:** `SELECT *` forces PostgreSQL to abandon the slim RAM Index-Only Scan and perform random disk Heap Fetches + TOAST reads for all 40 columns.
- **Senior Fix:** Project strictly needed columns: `SELECT user_id, total_cents FROM orders WHERE user_id = 42;`.

3. `COUNT(*)` vs `COUNT(column)` & `AVG(column)` Analytics Skew

- **The Bug:** `SELECT COUNT(discount_code) FROM orders;` returns `1,200` on a table with `50,000` orders.
- **Why:** `COUNT(*)` counts rows; `COUNT(col)` and `AVG(col)` **silently skip `NULL`s**. `AVG(bonus)` over `[100, NULL]` returns `100`, not `50`!
- **Senior Fix:** Use `COUNT(*)` for total rows and `AVG(COALESCE(bonus, 0))` when `NULL` means zero.

4. The `OFFSET 1,000,000` Deep Pagination Crash

- **The Bug:** `SELECT id, title FROM posts ORDER BY id DESC LIMIT 20 OFFSET 1000000;` takes `4.5s` and spikes CPU.
- **Why:** The database still fetches, sorts, and walks past all `1,000,020` rows before discarding the first million.
- **Senior Fix:** Keyset (Cursor) Pagination: `WHERE id < :last_seen_id ORDER BY id DESC LIMIT 20` (`0.4ms` constant time).

5. Why Massive `DELETE` Makes PostgreSQL Slower (MVCC Dead Tuples)

- **The Bug:** Deleting 40M old log rows makes subsequent `SELECT` queries *slower*, not faster.
- **Why:** PostgreSQL MVCC marks deleted rows as dead tuples on disk pages until `VACUUM` reclaims visibility maps.
- **Senior Fix:** Partition time-series tables by month and run `DROP TABLE logs_2025_01;` (instant metadata drop, zero bloat).

6. Why `SELECT COUNT(*)` Takes 12 Seconds on 100M Rows

- **The Bug:** Dashboard runs `SELECT COUNT(*) FROM events;` on every page load.
- **Why:** Under MVCC, different transactions see different live rows, forcing a full visibility scan instead of reading a single header integer.
- **Senior Fix:** Maintain an event counter table or read `reltuples` from `pg_class` for instant approximate counts.

7. Random `UUIDv4` Primary Key B-Tree Fragmentation

- **The Bug:** Insert throughput drops 8x once a `UUIDv4` primary-key table exceeds RAM.
- **Why:** Uniformly random 128-bit keys hit cold B-Tree leaf pages across disk, causing constant 50% page splits and WAL amplification.
- **Senior Fix:** Use time-ordered `UUIDv7` (`RFC 9562`) for sequential B-Tree appends + globally unique IDs.

8. TOCTOU Double-Spend Race Condition (`qty = -1`)

- **The Bug:** Reading `qty == 1` in app code and then running `UPDATE items SET qty = qty - 1` lets two concurrent requests buy the last item.
- **Senior Fix:** Atomic single-statement guard: `UPDATE items SET qty = qty - 1 WHERE id = $1 AND qty >= 1 RETURNING qty;` or `SELECT ... FOR UPDATE`.

Part 2 — Python Internals & Fintech Traps (#9–#15)

9. Why `256 is 256` Is `True`, But `257 is 257` Is `False`

- **The Bug:** Using `if user_id is target_id:` works in unit tests (`id=42`) and fails randomly in production (`id=9500`).
- **Why:** CPython pre-allocates singleton integer objects only for `[-5, 256]`. `is` compares memory addresses (`id(x)`), NOT values!
- **Senior Fix:** Always use `==` for value equality; reserve `is` strictly for `None` (`if x is None:`).

10. Mutable Default Arguments (`def add_item(x, items=]):`)

- **The Bug:** Calling `add_item("a")` then `add_item("b")` returns `["a", "b"]` across unrelated requests!
- **Why:** Default arguments are evaluated **once at `def` import time**, sharing the exact same list across every call.
- **Senior Fix:** Default to `items=None` and initialize `if items is None: items = []` inside the function.

11. Storing Money as `FLOAT` (`100.00 - 19.99 * 3 = 40.029999999999994`)

- **The Bug:** Binary IEEE-754 floats cannot represent `0.1` or `19.99` finitely in base-2, drifting ledgers by thousands over 1M transactions.
- **Senior Fix:** Store integer minor units (`amount_cents BIGINT`) or exact fixed-point `NUMERIC(19, 4)` / Python `Decimal("19.99")`.

12. Late-Binding Lambda Closures (`[lambda: i for i in range(3)]`)

- **The Bug:** Every callback returns `2` instead of `0, 1, 2`.
- **Why:** Closures look up `i` in the enclosing scope *when called*, after the loop finished at `i = 2`.
- **Senior Fix:** Bind at definition time: `[lambda i=i: i for i in range(3)]`.

13. Modifying a `dict` While Iterating (`RuntimeError` at 3 AM)

- **The Bug:** `for k, v in sessions.items(): if expired(v): del sessions[k]` crashes in production as soon as one session expires.
- **Why:** Mutating hash-table layout invalidates the active iterator pointer.
- **Senior Fix:** Iterate over a key snapshot `for k in list(sessions):` or build a clean dict comprehension.

14. JavaScript / TypeScript `Array.forEach(async ...)` Fire-and-Forget Bug

- **The Bug:** `users.forEach(async u => await save(u)); console.log("Done");` prints `"Done"` before a single user is saved!
- **Why:** `forEach` discards returned Promises and never awaits callbacks.
- **Senior Fix:** Sequential `for (const u of users) await save(u)` or parallel `await Promise.all(users.map(save))`.

15. Unbounded `asyncio.gather(*100_000_tasks)` OOM & Socket Exhaustion

- **Senior Fix:** Gate concurrency with `asyncio.Semaphore(50)` or `asyncio.TaskGroup` worker pools.

Part 3 — Distributed Systems, Cloud & Security Traps (#16–#25)

16. The 8-Microservices `99%` Availability Trap

- **The Math:** Chaining 8 synchronous microservices that each boast `99.0%` uptime yields $0.99^8 = 92.27\%$ availability (**28 hours of downtime per year** -> **670 hours per year**!) plus compounded p99 tail latency.
- **Senior Fix:** Start with a modular monolith; decouple cross-domain writes via async queues (`SQS` / `Kafka`) + local read caches.

17. Cache Stampede (50,000 Users Hit an Expired Redis Key at `00:00:00`)

- **The Bug:** Hot cache key expires -> 50,000 threads simultaneously miss Redis and DDoS PostgreSQL.
- **Senior Fix:** Singleflight request coalescing (1 worker recomputes while 49,999 wait on the in-memory promise) + probabilistic jittered TTLs.

18. Fixed-Interval Retry Loops = Self-Inflicted DDoS

- **The Bug:** `sleep(2)` retry on 100k mobile clients synchronizes them into hammer waves every 2.00 seconds so the API never recovers.
- **Senior Fix:** Exponential Backoff + Full Jitter: `sleep(random.uniform(0, min(cap, base * 2 ** attempt)))`.

19. Stripe-Grade Idempotency Keys (Zero Double Charges)

- **Senior Fix:** Client sends `Idempotency-Key: <uuid>` stored atomically with the payment state machine so network retries return the cached receipt instead of charging twice.

20. `git rm .env` Does NOT Erase Leaked API Keys

- **Why:** Every prior commit blob stays permanently public in `.git/objects` history; bots scrape leaked keys in under 4 seconds.
- **Senior Fix:** **Revoke/rotate the key immediately**, then scrub history with `git filter-repo` if needed.

21. Storing Session JWTs in Browser `localStorage`

- **Why:** Any single XSS snippet (`npm` supply-chain or third-party script) runs `localStorage.getItem('token')` and exfiltrates every user session.
- **Senior Fix:** Store session tokens in `HttpOnly; Secure; SameSite=Lax` cookies.

22. Default `Dockerfile` Running as `root` (UID 0)

- **Senior Fix:** Create a non-root user (`USER appuser`), use multi-stage distroless/slim builds, and drop Linux capabilities.

23. Kubernetes CPU `limits` Throttling Multi-Threaded Services

- **Why:** Linux CFS 100ms quota slices get burned in 12ms by 8 worker threads, freezing your pod for the remaining 88ms even when the node is 80% idle!
- **Senior Fix:** Set accurate CPU `requests`, alert on `container_cpu_cfs_throttled_seconds_total`, and avoid tight CPU limits on latency-critical services.

24. `HTTP 401 Unauthorized` vs `403 Forbidden`

- **Rule:** `401` = *"Who are you?"* (missing/invalid authentication credentials). `403` = *"I know who you are, and you lack permission"* (re-logging in won't help).

25. Vector Search (`RAG`) Without Hybrid BM25 Keyword Filtering

- **Why:** Dense cosine embeddings miss exact SKU codes, error IDs, and ticket numbers (`ERR_4092`).
- **Senior Fix:** Combine dense vector similarity + sparse `BM25` full-text search fused with Reciprocal Rank Fusion (`RRF`).

Ready to Level Up Further?

- **Get the Complete 30-Blueprint Senior System Design Playbook + 30 Runnable `.py` Scripts + 30 HD Flashcards ($19):** https://gabhack.github.io/mochacat-store/#pricing
- **Get the Complete Code & Chill Pro Bundle — Playbook + 50 Studio-Mastered DMCA-Safe Lo-Fi & City Pop MP3s ($29):** https://gabhack.github.io/mochacat-store/#pricing
- **Subscribe to MochaCat on YouTube for Daily Quizzes & Deep Focus Mixes:** https://www.youtube.com/channel/UCW6R0z_vBpQX0lqDLJOmslQ?sub_confirmation=1